

RENDERING STRIP — WHERE AND WHEN HTML IS PRODUCED				
Strategy	Runs	App Router	Pages Router	Use for
CSR	browser, after JS loads	'use client' + useEffect/SWR/TanStack Query; next/dynamic(..., {ssr:false}) inside a Client Component; output: 'export' SPA	useEffect, next/dynamic ssr:false	per-user widgets, browser-only APIs
SSR (dynamic)	server, every request	automatic when a route reads cookies() headers() searchParams, connection() or uncached data; f; in build table	getServerSideProps	personalised or always-fresh pages
SSG (static)	server, at next build	default for routes without request data; generateStaticParams for [id]; o / ●	getStaticProps + getStaticPaths	catalogue, docs, marketing
ISR	build, then in background	export const revalidate = 3600; fetch(u,{next:{revalidate}}), on demand revalidateTag/Path; CC: cacheLife	revalidate in getStaticProps; res.revalidate(); fallback	static pages that change
PPR	static shell at build + dynamic holes per request	cacheComponents: true; dynamic parts inside <Suspense>, cached parts 'use cache'; ● Partial Prerender	—	mostly-static pages with a personal slot
Streaming	server sends HTML in chunks	loading.tsx, <Suspense fallback>; status/headers fixed once the shell is sent	— (whole page at once)	slow data without blocking the shell

REACT: JSX, COMPONENTS, PROPS

```
export function BookCard({ book, onPick }: Props) {
  if (!book) return null; // nothing return
  return (
    <li onClick={() => onPick(book.id)}>
      <h3 className="title">{book.title}</h3>
      {book.rating > 4 && <Badge />}
      {book.tags.map(t =>
        <Tag key={t} name={t} />)}
    </li>);
}
```

One root (or <.../>), className/htmlFor, camelCase events, {expr}; stable unique key in lists; props are read-only, children is a prop. Components are pure functions of props + state.

PROJECT & CLI

```
npx create-next-app@latest bookshelf --yes
next dev # Turbopack, localhost:3000
next build # route table ● ● ● f
next start # production server
next upgrade # bump next/react (16.1)
next typegen # PageProps/LayoutProps types
next info | next telemetry disable
Defaults: TypeScript, ESLint (flat config), Tailwind v4,
App Router, Turbopack, @/* alias, AGENTS.md.
Config: next.config.ts exporting NextConfig.
```

STATE & EVENTS

```
const [count, setCount] = useState(0);
setCount(c => c + 1); // updater form
const [s, dispatch] = useReducer(fn, init);
<input value={q}
  onChange={e => setO(e.target.value)} />
```

State is a snapshot per render, updates are batched; never mutate (copy objects/arrays). Lift state up to share it, derive values instead of syncing state.

FILE CONVENTIONS (APP/)	
layout	shared UI, keeps state; root has <html><body>
page	makes a route public
loading	Suspense fallback
error/global-error	error boundary (client)
not-found/forbidden/unauthorized	404 / 403 / 401 UI
template	layout re-mounted per nav
default	parallel slot fallback
route	Route Handler (no page)
icon/opengraph-image	metadata files
sitemap/robots/manifest	generated SEO files
root/proxy.ts	runs before routes
instrumentation(-client).ts	observability hooks
Folders: [id] [...slug] [...slug] (group) _private @slot (.);x; public/ served at /.	

ENVIRONMENT VARIABLES

```
.env .env.local .env.development
.env.production .env.production.local
AUTH_SECRET=... # server only
NEXT_PUBLIC_SITE_URL=... # inlined at build
Only NEXT_PUBLIC.* reaches the browser, frozen at
next build; read runtime values on the server
(await connection() first). .env*.local not
committed; runtime config removed in 16.
```

ROUTING

```
// app/[locale]/books/[id]/page.tsx
export default async function Page({ params }: PageProps<"/[locale]/books/[id]> {
  const { id } = await params; // async (15)
  const book = await getBook(id);
  if (!book) notFound();
  return <Book book={book} />; }
[...slug] catch-all, [...slug]! optional; (shop)
group (not in URL, multiple root layouts); @modal
parallel slot + default.tsx; (.)(.) (...)
intercepting; generateStaticParams,
dynamicParams.
```

NAVIGATION

```
import Link from 'next/link';
<Link href={'/books/${id}'}>Dune</Link>
'use client';
import { useRouter, usePathname,
  useSearchParams, useParams,
  useSelectedLayoutSegment } from
  'next/navigation';
router.push('/books/'); router.replace(u);
router.refresh();
import { redirect, permanentRedirect,
  notFound } from 'next/navigation';
// 307 / 308 / 404
useLinkStatus for pending links; Pages Router uses
next/router. Instant Navigations (16.3):
cacheComponents + partialPrefetching.
```

SERVER VS CLIENT COMPONENTS

Server (default)	async, data/secrets access, zero JS shipped; no state/effects/browser APIs
Client 'use client';	state, effects, events, browser APIs; prerendered to HTML then hydrated

Boundary rules: 'use client' marks an entry point, everything it imports is client code; Server Components can be passed as children/props into Client ones; props crossing the boundary must be serializable (no functions except Server Actions); import 'server-only' guards secrets; keep providers and interactivity in small leaves.

HYDRATION

Sequence: server renders HTML + RSC payload → browser paints HTML → JS loads → hydrateRoot attaches events to Client Components (selectively, by suspense boundary, user input first) → interactive.

Mismatch cause	Fix
invalid nesting (<div> in <p>)	valid HTML
typeof window, localStorage in render	useEffect + state, useSyncExternalStore
Date.now(), Math.random(), locale/time zone	compute on server, pass as prop; fixed locale
random ids	useId
browser extensions, CDN rewrites	suppressHydrationWarning (one level)
browser-only widget	dynamic(..., {ssr:false}) in a Client Component

Errors: react-hydration-error, React #418; dev overlay shows a +Client/-Server diff (16.2).

DATA FETCHING

```
export default async function Books() {
  const books = await db.books.findMany();
  return <List books={books} />; } // Server
const [a, b] = await Promise.all([
  getA(), getB()]); // parallel
Fetch in Server Components close to where data is
used; React.cache dedupes per request; parallelise,
avoid waterfalls; stream slow parts with <Suspense>;
pass a promise to a Client Component and read it
with use. Client: SWR / TanStack Query. fetch is
uncached by default (15+).
```

SERVER ACTIONS

```
'use server';
export async function saveReview(
  prev: State, form: FormData) {
  const user = await requireUser(); // auth!
  const data = ReviewSchema.parse(
    Object.fromEntries(form));
  await db.reviews.create(
    { ...data, userId: user.id });
  updateTag('book-${data.bookId}');
  redirect('/books/${data.bookId}'); }
POST only, public endpoints: always validate (Zod)
and authorise; progressive enhancement with <form
action>; refresh(), revalidatePath; after() for
work after the response.
```

CACHING LAYERS

Cache Components (cacheComponents: true): nothing cached unless marked; variants 'use cache: private' (per user) and 'use cache: remote' (shared store).

```
async function getBook(id: string) {
  'use cache';
  cacheLife('hours');
  cacheTag('book-${id}');
  return db.books.find(id); }
```

cacheLife	stale	revalidate	expire
default	5 min	15 min	never
seconds	30 s	1 s	1 min
minutes	5 min	1 min	1 h
hours	5 min	1 h	1 day
days	5 min	1 day	1 week
weeks	5 min	1 week	30 days
max	5 min	30 days	1 year

Previous model: request memoization → Data Cache (fetch cache: 'force-cache', next: {revalidate, tags}, unstable_cache) → Full Route Cache (dynamic, revalidate segment config) → client route cache (stateTimes).

Revalidate: revalidateTag(tag, 'max') (state-while-revalidate; 1-arg form deprecated), updateTag(tag) and refresh() (Server Actions only), revalidatePath(path), router.refresh().

ROUTE HANDLERS & PROXY

```
// app/api/books/route.ts
export async function GET(req: NextRequest) {
  const q = req.nextUrl.searchParams.get('q');
  return Response.json(await search(q)); }
// proxy.ts (was middleware.ts; Node.js)
export function proxy(req: NextRequest) {
  return NextResponse.redirect(
    new URL('/en', req.url)); }
export const config = {
  matcher: ['/(?!api)_next.*') ];
Handlers: GET POST PUT PATCH DELETE, await
params, GET uncached by default (15+). Proxy:
redirects, rewrites, headers, optimistic auth only;
redirects()/rewrites()/headers() in config.
```

METADATA, IMAGES, FONTS

```
export const metadata: Metadata = {
  title: { template: '%s | Bookshelf',
    default: 'bookshelf' } };
export async function generateMetadata(
  { params } { /* per-page */ } ) {
  <Image src={cover} alt="Cover of Dune"
    width={300} height={450} priority />
  import { Inter } from 'next/font/google';
  const inter = Inter({ subsets: ['latin'] });
  Remote images need images.remotePatterns;
  next/script strategies; @next/third-parties; OG
  images with ImageResponse; sitemap.ts,
  robots.ts, JSON-LD in a <script>.
```

AUTH & DATA SECURITY

```
// auth.ts (Auth.js v5: next-auth[beta])
export const { handlers, auth, signIn,
  signOut } = NextAuth(
  { providers: [Github] });
// lib/dal.ts (AUTH_SECRET in env)
import 'server-only';
export const requireUser = cache(async () => {
  const s = await auth();
  if (!s?.user) unauthorized();
  return s.user; });
Check auth in the Data Access Layer, every page,
action and handler (Proxy is only optimistic); return
DTOs, not rows; forbidden()/unauthorized() need
authInterrupts; httpOnly Secure SameSite cookies;
CSP with nonces in Proxy; taint APIs; patch security
releases promptly.
```

TESTING

```
npm i -D vitest @vitejs/plugin-react jsdom \
  @testing-library/react @playwright/test
npx vitest # unit tests (sync comps)
npx playwright test # E2E: build + start
Async Server Components: test with E2E; Jest via
next/jest; Cypress; MSW to mock APIs; instant()
from @next/playwright (16.3).
```

DEPLOYMENT & OUTPUT MODES

next start	Node.js server, all features
output: 'standalone'	minimal server for Docker (node server.js)
output: 'export'	static output /, no server features

adapters (16.2 stable) Vercel, OpenNext, Netlify...

Self-hosting: shared cache handler for many instances, deployment/ID, image optimisation, instrumentation.ts + OpenTelemetry; Web Vitals: LCP, INP, CLS (useReportWebVitals).

TURBOPACK & TOOLING

Turbopack is the default for dev and build (16); --webpack to opt out; custom webpack() config fails the build unless chosen explicitly. DevTools, dev overlay, next dev --inspect, next experimental-analyze, Next DevTools MCP.

CHANGED SINCE OLDER TUTORIALS (BOOKS TARGET NEXT.JS 12-13.2, PAGES ROUTER)

13 App Router; Pages data functions (getServerSideProps...) → async Server Components	15 async params, searchParams, cookies(), headers(); 16 sync access removed	16 middleware.ts → proxy.ts (export proxy, Node.js)	14 next export removed → output: 'export'
16 next lint removed → ESLint CLI (flat config)	13 <Link> needs no <a>; legacyBehavior deprecated	images.domains deprecated → remotePatterns; next/legacy/image	NextAuth v4 → Auth.js v5 (auth(), AUTH_*, beta, now part of Better Auth)
Web Vitals FID → INP (March 2024)	15 fetch/GET handlers uncached; 16 revalidateTag(tag, profile), updateTag	16 Turbopack default; cacheComponents replaces experimental.ppr/dynamicIO	16 AMP, serverRuntimeConfig/publicRuntimeConfig removed

Next.js is a trademark of Vercel, Inc.; React is a trademark of Meta Platforms, Inc. This unofficial summary covers the official documentation (nextjs.org/docs, react.dev). Tested against Next.js 16.3.8 and React 19.3.0 on 2026-10-03. irurueta.com